

IF2211 Strategi Algoritma

**Pemanfaatan Algoritma BFS dan DFS dalam Pencarian Recipe pada
Permainan Little Alchemy 2**

Laporan Tugas Besar 2

Disusun untuk memenuhi tugas mata kuliah Strategi Algoritma pada Semester 4
(empat) Tahun Akademik 2024/2025



Disusun Oleh:

Brian Ricardo Tamin (13523126)

Nathanael Rachmat (13523142)

Muhammad Farrel Wibowo (13523153)

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

BANDUNG 2025

DAFTAR ISI

DAFTAR ISI.....	2
BAB I DESKRIPSI TUGAS.....	4
BAB II LANDASAN TEORI.....	6
2.1. Graf.....	6
2.2. BFS (Breadth First Search).....	6
2.3. DFS (Depth First Search).....	6
2.4. Aplikasi Web dan Teknologi yang Digunakan.....	7
2.4.1 Backend: Go (Golang).....	7
2.4.2 Frontend: Next.js, React, dan TypeScript.....	7
BAB III ANALISIS PEMECAHAN MASALAH.....	9
3.1. Langkah-langkah pemecahan masalah.....	9
3.2. Proses Pemetaan Masalah dan Penjelasan Algoritma.....	10
3.2.1 Pendekatan Algoritma BFS (Breadth First Search).....	10
3.2.1 Pendekatan Algoritma DFS (Depth First Search).....	11
3.3. Fitur fungsional dan arsitektur aplikasi web yang dibangun.....	12
3.4. Contoh Ilustrasi Kasus.....	13
3.4.1. Contoh Ilustrasi Kasus BFS.....	13
3.4.2. Contoh Ilustrasi Kasus DFS.....	13
BAB IV IMPLEMENTASI DAN PENGUJIAN.....	15
4.1 Library.....	15
4.2 Struktur.....	16
4.2.1 Backend.....	16
4.2.2 Frontend.....	17
4.3 Struktur Data.....	18
4.3.1 Struktur Data Tree.....	18
4.3.2 Struktur Data Graph.....	19
4.3.3. Struktur Data dan Fungsi BFS.....	21
4.3.4. Struktur Data dan Fungsi DFS.....	22
4.3.5. Struktur Data dan Fungsi Scraping.....	23
4.3.6. Fungsi Config.go.....	23
4.3.7. Struktur Data dan Fungsi main.go (Routing & API).....	24
4.4 Penjelasan tata cara penggunaan program.....	25
4.4.1. Interface Program.....	25
4.4.2. Fitur-Fitur yang Disediakan Program.....	28
4.4.3. Alur Penggunaan Program.....	29
4.5 Hasil pengujian.....	30
4.5.1. DFS One Recipe (Element: Grilled Cheese).....	30
4.5.2. DFS One Recipe (Element: Sky).....	30
4.5.3. BFS One Recipe (Element: Sky).....	31
4.5.4. DFS MultiRecipe (Element: Sky) dengan Jumlah Resep Dua.....	31

4.5.5. DFS MultiRecipe (Element: Sky) dengan Jumlah Resep Tiga.....	32
4.5.6. BFS MultiRecipe (Element: WaterGun).....	32
4.6 Analisis hasil pengujian.....	32
BAB V PENUTUP.....	34
5.1. Kesimpulan.....	34
5.2. Saran.....	34
5.3. Refleksi.....	34
LAMPIRAN.....	35
DAFTAR PUSTAKA.....	36

Little Alchemy 2 merupakan permainan berbasis web / aplikasi yang dikembangkan oleh Recloak yang dirilis pada tahun 2017, permainan ini bertujuan untuk membuat 720 elemen dari 4 elemen dasar yang tersedia yaitu air, earth, fire, dan water. Permainan ini merupakan sekuel dari permainan sebelumnya yakni Little Alchemy 1 yang dirilis tahun 2010.

Dalam tugas besar ini, mahasiswa diminta untuk:

- IF2211 Strategi Algoritma 4

Komponen-komponen dari permainan ini antara lain:

1. **Elemen dasar**

Dalam permainan Little Alchemy 2, terdapat 4 elemen dasar yang tersedia yaitu water, fire, earth, dan air, 4 elemen dasar tersebut nanti akan di-combine menjadi elemen turunan yang berjumlah 720 elemen.



Gambar 2. Elemen dasar pada Little Alchemy 2
(sumber: <https://www.thegamer.com>)

2. **Elemen turunan**

Terdapat 720 elemen turunan yang dibagi menjadi beberapa tier tergantung tingkat kesulitan dan banyak langkah yang harus dilakukan. Setiap elemen turunan memiliki recipe yang terdiri atas elemen lainnya atau elemen itu sendiri.

3. **Combine Mechanism**

Untuk mendapatkan elemen turunan pemain dapat melakukan combine antara 2 elemen untuk menghasilkan elemen baru. Elemen turunan yang telah didapatkan dapat digunakan kembali oleh pemain untuk membentuk elemen lainnya.

BAB II

LANDASAN TEORI

2.1. Graf

Permasalahan pencarian solusi pada permainan seperti *Little Alchemy 2* dapat direpresentasikan dalam bentuk graf, di mana simpul menyatakan elemen dan sisi menunjukkan hubungan kombinasi antar elemen. Untuk melakukan pencarian elemen dalam graf ini, digunakan dua algoritma traversal graf yang paling mendasar yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Keduanya termasuk dalam kategori algoritma pencarian tak terinformasi (uninformed/blind search), yakni algoritma yang tidak memiliki pengetahuan tambahan mengenai seberapa dekat suatu simpul terhadap solusi.

2.2. BFS (Breadth First Search)

BFS adalah metode pencarian yang melakukan penelusuran graf secara melebar dari simpul awal. Simpul-simpul yang berada pada tingkat kedalaman yang sama dikunjungi terlebih dahulu sebelum melanjutkan ke tingkat berikutnya. Traversal dilakukan menggunakan struktur data queue (antrian FIFO). Prosesnya dimulai dari simpul akar, lalu semua simpul yang bertetangga langsung dengannya dimasukkan ke dalam antrian, kemudian diikuti oleh simpul-simpul dari tetangganya, dan seterusnya. Hasil penelusuran BFS dapat divisualisasikan sebagai pohon ruang status (state space tree), di mana level demi level diekspansi secara urut.

BFS menjamin kelengkapan (completeness), artinya jika solusi ada maka pasti akan ditemukan. Selain itu, BFS juga optimal jika setiap langkah memiliki bobot yang sama, karena solusi pertama yang ditemukan adalah solusi dengan langkah minimum. Namun, algoritma ini memerlukan memori yang besar, dengan kompleksitas ruang dan waktu sebesar $O(b^d)$, di mana b adalah branching factor dan d adalah kedalaman solusi.

2.3. DFS (Depth First Search)

DFS adalah metode pencarian yang menelusuri graf secara mendalam dengan mengikuti satu cabang hingga tidak ada lagi simpul yang dapat dikunjungi, lalu kembali (backtrack) ke simpul sebelumnya untuk menjelajahi cabang lain yang belum dikunjungi. Traversal ini biasanya menggunakan stack (tumpukan LIFO) atau teknik rekursif. Sama seperti BFS, DFS membentuk pohon ruang status, namun simpul-simpul dalam cabang tunggal dieksplorasi sepenuhnya sebelum kembali ke cabang lain. DFS memiliki kompleksitas waktu dan ruang sebesar $O(b^m)$, dengan m adalah kedalaman maksimum graf. DFS tidak optimal karena bisa saja menemukan solusi yang sangat panjang terlebih dahulu, dan juga tidak selalu lengkap kecuali dengan penanganan terhadap lintasan berulang (redundant paths). Namun, DFS unggul dalam efisiensi ruang dan cocok digunakan untuk pencarian menyeluruh, eksplorasi semua solusi, dan implementasi teknik backtracking.

2.4. Aplikasi Web dan Teknologi yang Digunakan

Aplikasi pencarian *recipe* elemen dalam permainan *Little Alchemy 2* dikembangkan dengan pendekatan arsitektur web berbasis client-server. Dalam model ini, backend bertugas menyediakan data dan layanan melalui API, sedangkan frontend bertugas menyajikan antarmuka pengguna dan melakukan logika tampilan. Komunikasi antara keduanya dilakukan melalui HTTP menggunakan format data JSON. Teknologi yang digunakan mencakup Go (Golang) di sisi backend, serta React, Next.js, dan TypeScript di sisi frontend. Selain itu, digunakan pula Docker untuk kebutuhan kontainerisasi aplikasi.

2.4.1 Backend: Go (Golang)

Bahasa Go memiliki kemampuan dalam menangani proses concurrent secara efisien melalui goroutine dan channel. Bahasa ini sangat sesuai untuk proses scraping data secara paralel serta implementasi algoritma BFS dan DFS yang membutuhkan performa tinggi dalam traversing graph. Backend yang dibangun menggunakan Go berfungsi untuk melakukan scraping elemen dan recipe dari situs wiki *Little Alchemy 2*, menjalankan proses pencarian, dan menyediakan endpoint API yang akan dikonsumsi oleh frontend.

2.4.2. Frontend: Next.js, React, dan TypeScript

Frontend aplikasi dibangun menggunakan React dengan dukungan framework Next.js sebagai kerangka kerja pendukung untuk modularisasi dan pengelolaan halaman. Seluruh rendering dilakukan secara Client-Side Rendering (CSR), di mana halaman dan konten dirender di sisi klien setelah JavaScript termuat sepenuhnya di browser. Pendekatan ini dipilih karena lebih sesuai untuk aplikasi interaktif seperti visualisasi pohon recipe yang memerlukan pembaruan cepat berdasarkan input pengguna.

Seluruh proses algoritmik, seperti pencarian jalur menggunakan BFS dan DFS, dilakukan sepenuhnya di backend (Go). Frontend hanya bertugas mengirim permintaan HTTP POST ke endpoint seperti `/api/bfs`, `/api/dfs`, dan endpoint pencarian jalur terpendek. Backend kemudian mengembalikan hasil pencarian dalam format JSON, yang divisualisasikan di frontend menggunakan pustaka seperti React Flow.

Penggunaan TypeScript pada frontend memperkuat struktur dan integritas kode dengan menjamin keamanan tipe data (type safety), meningkatkan dokumentasi otomatis, serta memudahkan kolaborasi dan debugging. Hal ini sangat penting

mengingat komunikasi antara frontend dan backend bergantung pada struktur data yang kompleks dari hasil algoritma yang dijalankan di server.

2.4.3. Kontainerisasi dengan Docker

Docker digunakan untuk membungkus aplikasi backend dan frontend dalam container yang terpisah. Dengan kontainerisasi, seluruh lingkungan aplikasi termasuk dependensi, runtime, dan konfigurasi dapat dijalankan secara konsisten pada berbagai mesin tanpa perlu setup ulang. Docker mendukung deployment yang mudah dan cepat, serta sangat berguna untuk integrasi sistem dalam lingkungan tim. Dengan satu konfigurasi Dockerfile dan docker-compose.yml, pengembang dapat menjalankan aplikasi lintas platform dengan satu perintah.

BAB III

ANALISIS PEMECAHAN MASALAH

3.1. Langkah-langkah pemecahan masalah

Penyelesaian persoalan pencarian *recipe* elemen pada permainan *Little Alchemy 2* dimulai dengan proses web scraping terhadap halaman wiki yang memuat data elemen dan kombinasi pembentuknya. Hasil scraping ini kemudian disusun ke dalam sebuah list of struct dengan tipe Element, yang memuat nama elemen (Name), daftar kombinasi pembentuknya (Recipes), dan tingkat kedalaman atau kesulitannya (Tier). Representasi awal ini bersifat sederhana dan linear, serta mencerminkan struktur data yang mentah dari hasil parsing HTML.

Setelah data elemen terkumpul, struktur list tersebut dikonversi ke dalam bentuk map menggunakan fungsi ToScrapedMapWithTiers. Fungsi ini menyusun data dalam bentuk pemetaan antara nama elemen dengan struct ScrapedData yang memuat daftar ingredient (komponen pembentuk) dan tier-nya. Konversi ke map dilakukan untuk mempermudah akses langsung berdasarkan nama elemen, sekaligus mempersiapkan data agar dapat diubah ke representasi graf.

Langkah selanjutnya adalah membentuk graf elemen (ElementGraph) yang merepresentasikan keterkaitan antar elemen dalam bentuk arah (directed). Setiap node pada graf adalah struct ElementGraph yang menyimpan nama elemen, daftar recipe yang dimiliki (Recipes), serta informasi elemen lain yang menggunakan elemen tersebut sebagai bahan (UsedIn). Masing-masing recipe dalam graf direpresentasikan dengan struct Recipe, yang memuat referensi ke tiga node: dua elemen pembentuk (FirstElement dan SecondElement) serta hasil kombinasinya (ResultElement). Dengan membentuk graf ini, hubungan antar elemen menjadi eksplisit dan traversal antar elemen dapat dilakukan dengan efisien.

Setelah graf dibentuk dan setiap node dihubungkan melalui recipe-nya, proses pencarian recipe dari elemen dasar menuju elemen target dilakukan dengan algoritma Breadth First Search (BFS) atau Depth First Search (DFS). Algoritma traversal ini digunakan untuk membentuk pohon solusi (solution tree) yang merepresentasikan jalur kombinasi elemen dari elemen-elemen dasar hingga mencapai elemen target. Pohon ini akan divisualisasikan di sisi frontend dan menjadi hasil utama dari aplikasi, yang menunjukkan bagaimana sebuah elemen dapat dibuat berdasarkan urutan recipe yang valid.

3.2. Proses Pemetaan Masalah dan Penjelasan Algoritma

seluruh elemen hasil scraping dari situs wiki Little Alchemy 2 diolah menjadi struktur graf berarah. Dalam graf ini, setiap simpul mewakili satu elemen, dan sisi menghubungkan dua elemen pembentuk (ingredient) ke hasil kombinasinya. Dengan demikian, permasalahan dapat dimodelkan sebagai traversal graf dinamis, di mana pencarian dilakukan dari elemen target secara mundur menuju elemen-elemen dasar.

Pemetaan ke algoritma traversal dilakukan dengan membentuk representasi graf `ElementGraph`, yang menyimpan informasi tentang resep suatu elemen dan keterkaitannya dengan elemen lain. Proses traversal dilakukan menggunakan dua algoritma utama: Breadth First Search (BFS) dan Depth First Search (DFS). Tujuannya adalah membangun sebuah pohon solusi (solution tree) yang menunjukkan jalur pembentukan elemen secara eksplisit.

3.2.1 Pendekatan Algoritma BFS (*Breadth First Search*)

Algoritma Breadth First Search (BFS) digunakan untuk menjelajahi graf secara melebar dari elemen target, membentuk pohon kombinasi resep berdasarkan urutan tingkat (level-wise). Berikut adalah tahapan yang dilakukan dalam implementasi program:

- **Inisialisasi Akar Pohon**

Program memulai dengan membuat simpul akar (`TreeNodeElement`) yang mewakili elemen target. Simpul ini menjadi titik awal dari pohon solusi.

- **Memasukkan Simpul ke Queue**

Simpul akar dan elemen target dimasukkan ke dalam struktur data queue untuk antrian pemrosesan. Queue digunakan untuk menjamin bahwa simpul pada tingkat yang lebih tinggi (lebih dekat ke target) diproses terlebih dahulu.

- **Proses Traversal Level-wise**

Selama queue tidak kosong, program akan:

- Mengeluarkan simpul dari antrian.
- Memeriksa seluruh recipe dari elemen terkait.
- Untuk setiap recipe yang valid berdasarkan tier:
 - Buat dua simpul anak dari elemen pembentuk.
 - Hubungkan keduanya ke parent melalui `TreeNodeRecipe`.
 - Tambahkan anak-anak tersebut ke antrian untuk diproses selanjutnya.

- **Pengecekan Jalur Lengkap**

Jika kedua anak dari sebuah recipe adalah leaf (elemen dasar seperti Air, Earth, Water, atau Fire), maka tidak langsung dianggap sebagai solusi. Program melakukan pengecekan ulang apakah jalur tersebut benar-benar lengkap dengan menggunakan fungsi `isRecipeSolved()` dan `checkFullPath()`.

Jika belum lengkap (misalnya salah satu leaf sebenarnya belum benar-benar mencapai dasar), pencarian akan dilanjutkan sampai jalur tersebut valid.

- **Live Update dan Multithreading**

Pada mode multiple recipe, setiap recipe diproses dalam goroutine, dan setelah penambahan node baru, salinan pohon dikirim ke channel updates untuk ditampilkan di frontend secara real-time (fitur Live Update).

- **Pemotongan Jalur Tidak Lengkap**

Setelah traversal selesai, fungsi `pruneTreeToBasicPaths()` dijalankan untuk membersihkan cabang-cabang yang tidak mencapai elemen dasar secara sempurna.

- **Hasil Akhir**

Program mengembalikan `BFSResult` yang berisi pohon solusi, waktu eksekusi, jumlah node yang dikunjungi, dan jumlah jalur lengkap yang ditemukan.

3.2.1 Pendekatan Algoritma DFS (*Depth First Search*)

Selanjutnya DFS digunakan untuk menelusuri graf secara mendalam, memprioritaskan satu jalur hingga selesai sebelum mundur (backtrack). Langkah-langkah dalam implementasi DFS adalah sebagai berikut:

- **Inisialisasi Akar**

Program membuat simpul root dari elemen target sebagai awal traversal.

- **Traversal Rekursif**

Fungsi `FindMultiplePath` atau `FindFirstPath` memproses setiap recipe dari elemen saat ini:

- Memastikan tier anak tidak lebih tinggi dari parent.
- Membentuk dua node anak dari elemen pembentuk recipe.
- Menambahkan `TreeNodeRecipe` ke node parent.
- Menjalankan rekursi pada anak pertama lalu anak kedua.

- **Pengecekan Jalur Lengkap Setelah Mencapai Leaf**

Jika kedua anak adalah leaf, program tidak langsung mencatat jalur tersebut sebagai solusi. Fungsi `isRecipeSolved` akan memeriksa keabsahan jalur dari leaf sampai root menggunakan rekursi untuk memastikan bahwa jalur tidak hanya terlihat lengkap, tetapi benar-benar valid berdasarkan struktur pohon.

- **Multithreading dan Sinkronisasi**

Sama seperti BFS, DFS juga menggunakan goroutine, `WaitGroup`, dan `Mutex` untuk mengelola eksekusi paralel dan mencegah race condition.

- **Early Stop untuk Shortest Path**

Pada mode pencarian recipe terpendek, DFS akan berhenti begitu menemukan satu jalur lengkap pertama menggunakan break.

- **Pemangkasan dan Pengembalian Hasil**

Tree yang telah terbentuk akan dipangkas untuk menghapus jalur tidak lengkap, dan hasil akhir berupa DFSResult dikembalikan.

3.3. Fitur fungsional dan arsitektur aplikasi web yang dibangun.

Aplikasi web yang dikembangkan dalam tugas besar ini dirancang untuk membantu pengguna menemukan jalur kombinasi (recipe) dalam permainan Little Alchemy 2 dengan pendekatan yang interaktif dan fleksibel. Dari sisi fitur fungsional, pengguna dapat memasukkan nama elemen target, lalu memilih algoritma pencarian yang ingin digunakan, yaitu Breadth First Search (BFS) atau Depth First Search (DFS). Aplikasi menyediakan dua mode pencarian utama, yaitu shortest path (pencarian satu jalur terpendek) dan multi-recipe (pencarian beberapa jalur sekaligus). Jika pengguna memilih mode multi-recipe, tersedia pengaturan jumlah maksimum jalur (maxPaths) yang ingin ditemukan.

Selain itu, aplikasi juga mendukung fitur Live Update, yaitu visualisasi real-time dari proses pencarian solusi dalam bentuk pohon. Jika fitur ini diaktifkan, pengguna dapat mengatur delay waktu (dalam milidetik) untuk melihat bagaimana algoritma membentuk jalur demi jalur secara bertahap, seperti simulasi traversal graf. Seluruh hasil pencarian, termasuk jumlah simpul yang dikunjungi, waktu eksekusi, dan pohon kombinasi resep, ditampilkan secara visual di antarmuka aplikasi.

Dari sisi arsitektur, aplikasi ini menggunakan pendekatan client-server, dengan frontend dibangun menggunakan React dan Next.js dengan pendekatan Client-Side Rendering (CSR). Seluruh antarmuka pengguna, pemilihan algoritma, pengisian parameter, dan visualisasi dijalankan sepenuhnya di browser. Frontend ditulis dalam TypeScript untuk menjamin keamanan tipe dan kemudahan pengembangan antarmuka interaktif.

Bagian backend dibangun menggunakan bahasa Go (Golang), yang bertanggung jawab atas proses utama, seperti scraping data elemen, membangun struktur graf, serta menjalankan algoritma BFS dan DFS untuk pencarian solusi. Komputasi dilakukan sepenuhnya di sisi server (server-side computation), baik dalam mode normal (REST API) maupun streaming visualisasi (WebSocket). Frontend berkomunikasi dengan backend melalui HTTP API dan WebSocket API, dan backend akan mengembalikan struktur pohon hasil pencarian dalam format JSON atau streaming update secara real-time.

Untuk keperluan deployment, aplikasi ini dikemas dalam container Docker, baik untuk komponen backend maupun frontend. Kedua container image tersebut kemudian di-*push* ke Docker Hub, sehingga dapat diakses secara publik oleh platform deployment. Proses deployment dilakukan menggunakan Northflank, sebuah platform cloud-based yang

menyediakan infrastruktur hosting berbasis container. Dengan arsitektur ini, aplikasi dapat diakses secara daring melalui internet, tetap stabil tanpa ketergantungan pada lingkungan lokal pengguna, serta mendukung skalabilitas dan kolaborasi pengembangan lintas sistem operasi.

3.4. Contoh Ilustrasi Kasus

3.4.1. Contoh Ilustrasi Kasus BFS

Pada pendekatan BFS , alur algoritma dibagi menjadi 2, yaitu pendekatan untuk mencari single path recipe menggunakan fungsi FindFirstPath() dan multiple recipe berdasarkan angka input user menggunakan fungsi FindMultiplePath(). Pada contoh kasus pencarian single path , jika user menginput suatu recipe misalkan “Planet” akan dibuat root node Planet dalam struktur Tree, kemudian digunakan queue untuk menjelajahi seluruh elemen dari graphMap hasil scraping, setiap node akan dicek apakah memiliki kombinasi resep (Recipe) yang valid. Jika valid buat node kiri & kanan dari recipe (child nodes), dan disambungkan ke parent, jika child adalah leaf node (Fire, Water, Earth, Air) dan semua elemen terpenuhi, dianggap path lengkap. Jalur tree yang tidak terpakai akan dibuang oleh prunePath(). Tree hasil akhir direturn melalui BFSResult, termasuk nodeCount, execution time, dan completePaths =1.

Pada pendekatan Multiple path menggunakan FindMultiplePath(), akan dicari kombinasi recipe dalam satu tree hingga sebanyak maxPaths. algoritma akan dimulai dengan menginisialisasi , membuat node tree dari input target, misal “Planet”. queue berisi pasangan elemen dan node pohonnya (bfsItem). Setiap node, semua resep valid (tier lebih rendah dari parent) dan disambung ke parent sebagai TreeNodeRecipe, jika updates tidak nil, copy tree dan kirim ke channel, jika elemen bukan leaf, push ke queue. Semua goroutine ditunggu dengan WaitGroup untuk setiap level iterasi. Semua kemungkinan kombinasi tree dibentuk. Jika jumlah branch melebihi maxPaths, dibuang dengan cutChildren(). Tree hasil akhir direturn dalam BFSResult yang berisi jumlah path, nodeCount, dll.

3.4.2. Contoh Ilustrasi Kasus DFS

Pada pendekatan DFS , alur algoritma dibagi menjadi 2, yaitu pendekatan untuk mencari single path recipe menggunakan fungsi FindFirstPath() dan multiple recipe berdasarkan angka input user menggunakan fungsi FindMultiplePath(). Untuk FindFirstPath misalkan kita menggunakan “Sky” sebagai recipe yang ingin dicari. Algoritma akan membuat root node TreeNodeElement yang merepresentasikan elemen target yang dicari resepnya. Kemudian membuat Tree berisi root node untuk menyimpan struktur hasil DFS sebagai Tree. inisialisasi nodeCount dan found sebagai atomic boolean , dimana found dipakai untuk menghentikan pencarian setelah path pertama ditemukan, kemudian DFS rekursif untuk membangun path pertama, return DFSResult berisi hasil berupa tree, jumlah node, waktu eksekusi, dan tanda selesai.

Disisi lain, pada pencarian multiple recipe menggunakan DFS digunakan FindMultipleRecipe() . misalkan input user adalah “Sky”. Algoritma dimulai dengan membuat basis found == true sebagai return pertama jika sudah ditemukan path yang diinginkan. lalu ditambahkan nodeCount untuk setiap iterasi. iterasi dilakukan menggunakan current.Recipes untuk mencari cara membuat current element. Dilakukan pengecekan tier resep, memastikan kedua child tidak melebihi tier parent dalam pembuatan tree. lalu ditambahkan ke node.Children menggunakan append . Lalu dikirim update melalui channel updates untuk websocket. rekursif dilakukan ke dfsFirst(left) , dan dfsFirst(right). DFS akan berhenti setelah path pertama ditemukan. Pada multiple recipe DFS juga digunakan mekanisme penghitungan branch untuk membuang children jika jumlah cabang melebihi jumlah max path yang diinginkan oleh user.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

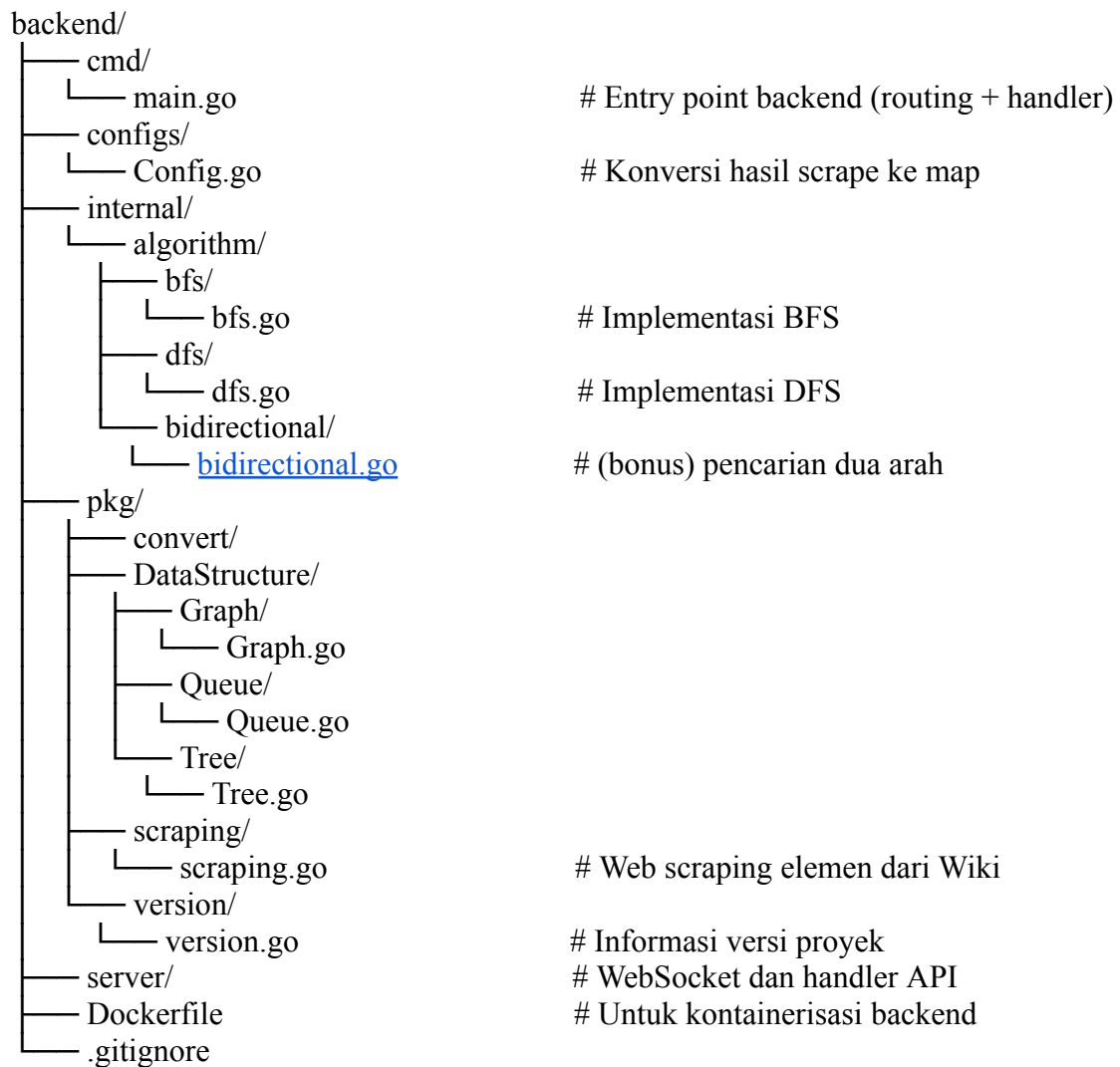
Tugas besar ini diimplementasikan menggunakan arsitektur client-server, dengan Go sebagai backend dan React + Next.js (TypeScript) sebagai frontend. Backend menangani scraping data, pembentukan graf, serta eksekusi algoritma BFS dan DFS dalam dua mode: *shortest path* dan *multi-recipe*, lengkap dengan fitur Live Update menggunakan WebSocket.

4.1 Library

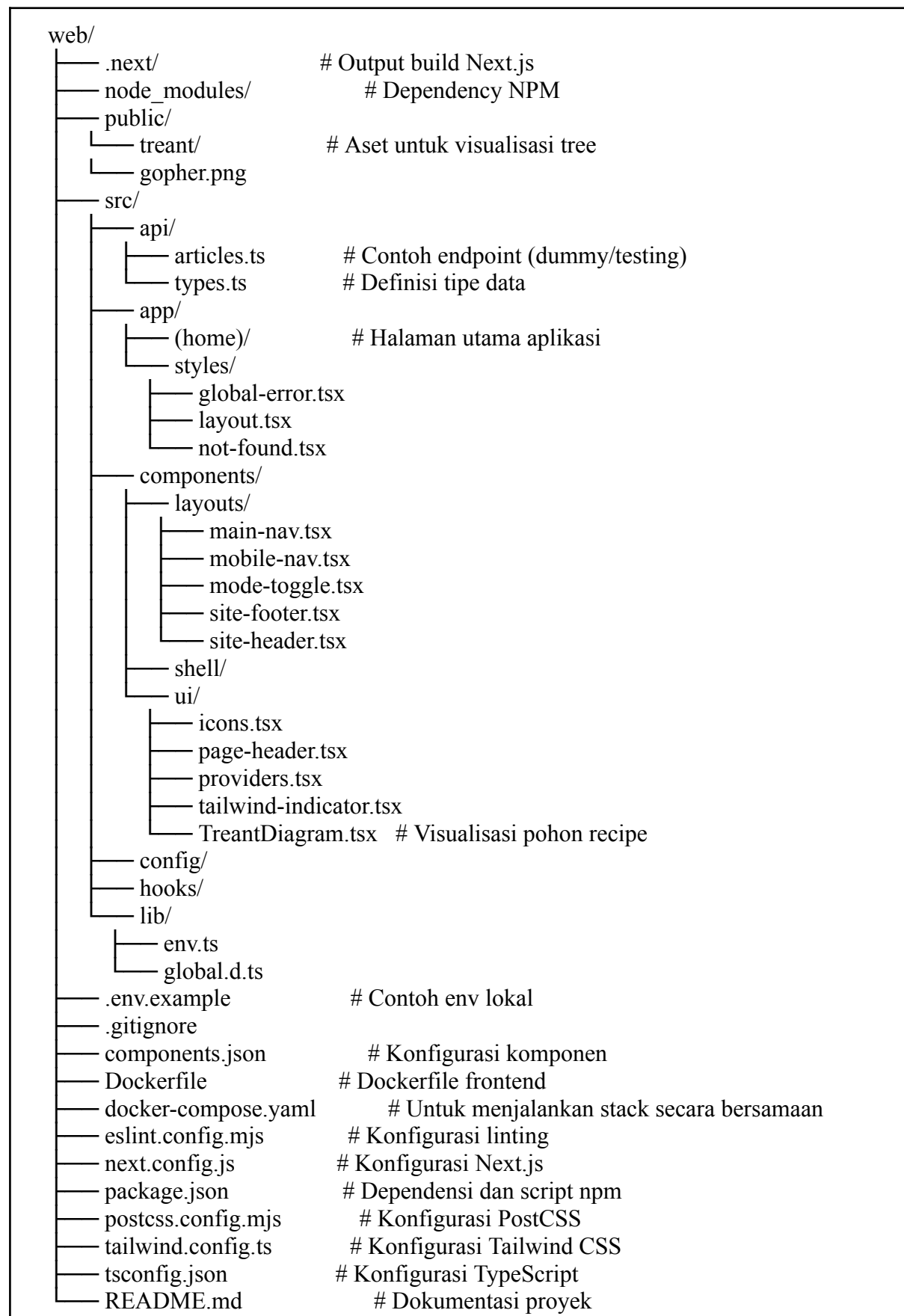
No.	Library	Fungsi Utama	Deskripsi Penggunaan
1	encoding/json	Serialisasi JSON	Mengubah data dari struct ke format JSON dan sebaliknya, digunakan untuk input/output API.
2	io	Operasi Sistem I/O	Membaca stream data seperti body request dengan <code>io.ReadAll</code> .
3	log	Logging	Menampilkan pesan error atau status server ke konsol dengan <code>log.Println</code> dan <code>log.Fatal</code> .
4	net/http	HTTP server & routing	Menangani routing dan pembuatan REST API, seperti <code>http.HandleFunc</code> dan <code>http.ListenAndServe</code> .
5	os	Interaksi System	Mengambil variabel lingkungan (contoh: <code>os.Getenv("PORT")</code>) untuk konfigurasi server.
6	sync	Menggunakan Thread	Menggunakan mutex untuk memanipulasi thread menggunakan group wait, sync, lock, dan unlock.
7	time	Pengelolaan Waktu	Mengatur delay (<code>time.Sleep</code>) dan mengukur waktu eksekusi (<code>time.Since</code>).

4.2 Struktur

4.2.1 Backend



4.2.2 Frontend



4.3 Struktur Data

4.3.1 Struktur Data Tree

Struct: Tree

Atribut/Metode	Tipe	Penjelasan
First	*TreeNodeElement	Simpul akar (root) dari pohon, merepresentasikan elemen target.
NewTree()	constructor	Membuat pohon baru dengan simpul akar dari elemen target.
MakeTree()	method	Menambahkan satu kombinasi resep (dua elemen) ke simpul parent.

Struct: TreeNodeElement

Atribut/Metode	Tipe	Penjelasan
Name	string	Nama elemen di simpul ini.
Children	[]TreeNodeRecipe	Daftar resep (kombinasi dua elemen) yang menjadi anak dari node ini.
Parent	*TreeNodeRecipe	Simpul parent yang menunjuk ke node hasil dari kombinasi (jika ada).

Struct: TreeNodeRecipe

Atribut/Metode	Tipe	Penjelasan
FirstElement	*TreeNodeElement	Elemen pertama dari sebuah recipe.
SecondElement	*TreeNodeElement	Elemen kedua dari sebuah recipe.

ResultElement	*TreeNodeElement	Elemen hasil dari kombinasi, menunjuk kembali ke parent node.
---------------	------------------	---

Fungsi

Fungsi	Penjelasan
CountNodes(*Tree)	Menghitung total node elemen dalam satu pohon resep.
CopyTree(*Tree)	Membuat salinan rekursif dari pohon (digunakan untuk Live Update).
PrintTree()	Menampilkan struktur pohon secara visual ke terminal (dengan indentasi).
PruneTreeToBasicPaths()	Menghapus cabang-cabang yang tidak berujung pada elemen dasar (Air/Earth/etc).
CountPaths()	Mengembalikan jumlah semua jalur kombinasi lengkap dari elemen dasar ke target.
isBasic(string)	Mengecek apakah elemen adalah elemen dasar (Air, Water, Earth, Fire).
copyElem(orig *TreeNodeElement, parentRecipe *TreeNodeRecipe)	digunakan untuk melakukan deep copy atau salinan menyeluruh terhadap sebuah pohon TreeNodeElement beserta seluruh turunannya.

4.3.2 Struktur Data Graph

Struct: ElementGraph

Atribut	Tipe	Penjelasan
Name	string	Nama elemen yang direpresentasikan oleh simpul graf.

UsedIn	[]Recipe	Daftar recipe di mana elemen ini digunakan sebagai bahan pembentuk.
Recipes	[]Recipe	Daftar recipe yang menghasilkan elemen ini.
Tier	int	Tingkat kedalaman elemen (semakin dasar semakin kecil nilainya).

Struct: Recipe

Atribut	Tipe	Penjelasan
FirstElement	*ElementGraph	Elemen pertama dalam kombinasi recipe.
SecondElement	*ElementGraph	Elemen kedua dalam kombinasi recipe.
ResultElement	*ElementGraph	Elemen hasil dari kombinasi recipe.

Fungsi-fungsi

Fungsi	Penjelasan
CreateElementGraphMap()	Membangun peta graf dari hasil scraping: node bahan, node hasil, dan recipe antar node.
GetBasicElement()	Mengembalikan array dari elemen dasar: Air, Earth, Water, dan Fire.
IsLeaf()	Mengembalikan true jika node adalah elemen dasar, false jika bukan.
Recipe.String()	Mengembalikan representasi string dari satu recipe (misal: "Water + Fire → Steam").

4.3.3. Struktur Data dan Fungsi BFS

Struct: BFSResult

Atribut	Tipe	Penjelasan
Tree	*Tree.Tree	Pohon solusi hasil pencarian BFS.
NodeCount	int	Jumlah node yang dikunjungi selama proses pencarian.
CompletePaths	int	Jumlah jalur solusi lengkap yang ditemukan.
ExecutionTimeMs	int64	Durasi eksekusi dalam milidetik.
Done	bool	Status apakah proses pencarian telah selesai.

Fungsi-fungsi BFS

Fungsi	Penjelasan
FindFirstPath()	Fungsi BFS utama yang mencari berbagai jalur kombinasi menggunakan pendekatan level-per-level secara paralel. Proses ini membentuk tree dan memperbarui hasil sementara ke frontend.
FindMultiplePath()	Fungsi BFS utama yang mencari berbagai jalur kombinasi menggunakan pendekatan level-per-level secara paralel. Proses ini membentuk tree dan memperbarui hasil sementara ke frontend.
pureBfs()	Implementasi BFS iteratif yang menjelajahi elemen secara mendalam hingga ditemukan satu jalur kombinasi yang valid. Digunakan oleh FindFirstPath.
prunePath()	Memangkas cabang-cabang pohon yang tidak menghasilkan jalur ke elemen dasar.
checkFullPath()	Memastikan bahwa sebuah jalur menuju target terdiri dari elemen dasar yang valid.
isRecipeSolved()	Mengecek apakah dua elemen pembentuk dari suatu node sudah tersolusikan.

isSolvedElem()	Cek rekursif apakah simpul sudah tersolusikan dari elemen dasar.
recalcBranchCount()	Menghitung ulang jumlah kombinasi jalur yang valid dari sebuah node ke elemen-elemen dasar setelah proses pemangkasan dilakukan.
cutChildren()	Memangkas jumlah anak (recipe) pada suatu node jika total jalur melebihi batas maxPaths. Proses pruning ini memastikan efisiensi dan membatasi eksplorasi berlebih.

4.3.4. Struktur Data dan Fungsi DFS

Struct: DFSResult

Atribut	Tipe	Penjelasan
Tree	*Tree.Tree	Pohon solusi hasil pencarian DFS.
NodeCount	int	Jumlah node yang dikunjungi selama proses pencarian.
CompletePaths	int	Jumlah jalur solusi lengkap yang ditemukan.
ExecutionTimeMs	int64	Durasi eksekusi dalam milidetik.
Done	bool	Status apakah proses pencarian telah selesai.

Fungsi-fungsi DFS

Fungsi	Penjelasan
FindShortestPath()	Mencari satu jalur tercepat menuju elemen target menggunakan DFS satu jalur.
dfsShortest()	DFS rekursif untuk pencarian shortest path, berhenti setelah satu solusi ditemukan.
FindMultiplePathDree()	Fungsi utama untuk menjalankan pencarian multi-recipe menggunakan DFS dengan multithreading. Membentuk Tree hasil pencarian dan menghitung total jalur kombinasi dari node target ke elemen dasar.
dfsMultiThreadingDree()	DFS rekursif multithread yang membentuk pohon solusi dengan penggunaan goroutine, mutex, dan waitgroup untuk sinkronisasi antar thread. Juga menangani caching hasil DFS untuk elemen dasar.

CopySubtree()	Melakukan deep copy pada subtree dari sebuah node TreeNodeElement, digunakan saat mengirim snapshot pohon hasil pencarian ke frontend melalui channel update.
cutChildren()	Memangkas jumlah anak (recipe) pada suatu node jika total jalur melebihi batas maxPaths. Proses pruning ini memastikan efisiensi dan membatasi eksplorasi berlebih.
min(a, b int) int	Fungsi utilitas untuk mengambil nilai minimum dari dua bilangan bulat. Digunakan dalam perhitungan pruning.

4.3.5. Struktur Data dan Fungsi Scraping

Struct: Element

Atribut	Tipe	Penjelasan
Name	string	Nama elemen hasil scraping dari halaman wiki.
Recipes	[][]string	Kumpulan kombinasi pasangan elemen yang menghasilkan elemen ini.
Tier	int	Tingkat elemen berdasarkan urutan kemunculannya.

Fungsi-fungsi Scraping

Fungsi	Penjelasan
ScrapeDatafromWeb()	Melakukan scraping dari halaman wiki dan menghasilkan slice of Element yang berisi nama, recipe, dan tier.
filterElements()	Membuang elemen yang tidak relevan atau tidak memiliki kombinasi resep yang valid.
removeInvalidRecipes()	Membuang kombinasi recipe yang menggunakan elemen yang tidak valid atau tidak ditemukan.

4.3.6. Fungsi [Config.go](#)

Fungsi-fungsi Configs

Fungsi	Penjelasan
--------	------------

ToScrapedMapWithTiers(elems []scraping.Element)	Mengonversi data hasil scraping menjadi peta elemen yang berisi bahan-bahan dan tier masing-masing.
---	---

4.3.7. Struktur Data dan Fungsi main.go (Routing & API)

Struct: request

Atribut	Tipe	Penjelasan
Target	string	Nama elemen yang akan dicari recipe-nya.
MaxPaths	int	Jumlah maksimum recipe yang ingin ditemukan.
DelayMs	int	Delay dalam milidetik antara tiap langkah pencarian (untuk live update).
ElementJSONPath	string	Path ke file JSON elemen jika digunakan.

Fungsi-fungsi API & Routing

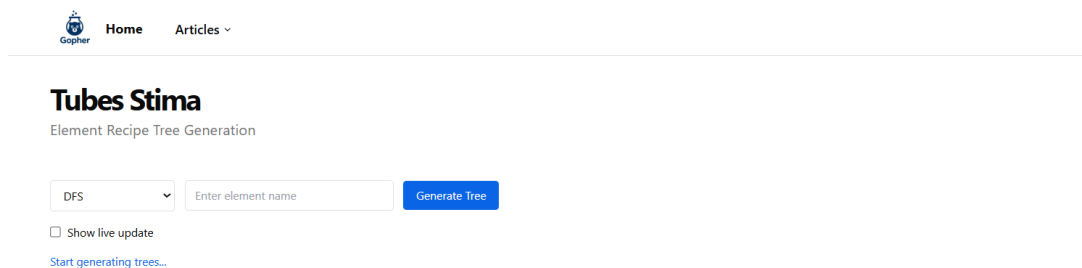
Fungsi	Penjelasan
main()	Menginisialisasi environment server, route handler REST dan WebSocket, serta menjalankan server.
configHandler()	Meng-handle endpoint /api/config untuk scraping elemen dan membuat graph map.
bfsHandler()	Endpoint POST /api/bfs, memproses permintaan pencarian multi-recipe menggunakan BFS.
dfsHandler()	Endpoint POST /api/dfs, memproses permintaan pencarian multi-recipe menggunakan DFS.
shortestBFShandler()	Endpoint POST /api/shortest/bfs, mencari satu recipe tercepat dengan BFS.
shortestDFShandler()	Endpoint POST /api/shortest/dfs, mencari satu recipe tercepat dengan DFS.
respondJSON()	Fungsi utilitas untuk mengubah hasil pencarian menjadi format JSON dan mengirimkannya.

withCORS()	Fungsi wrapper untuk mengaktifkan CORS header pada semua response endpoint.
normalizeName()	Fungsi utilitas untuk mengubah nama elemen menjadi kapitalisasi yang seragam.

4.4 Penjelasan tata cara penggunaan program

4.4.1. Interface Program

Program ini disajikan melalui antarmuka web berbasis React dan Next.js yang berjalan di sisi klien (Client Side Rendering). Namun, seluruh proses pencarian resep, pengambilan data, dan scraping dilakukan secara server-side menggunakan backend berbasis bahasa Go.



Gambar 2. Antarmuka Website

Tampilan antarmuka terdiri dari beberapa bagian utama:

- Form Input:
 - Target Element: Input teks untuk menentukan elemen akhir yang ingin dicari, misalnya "Brick", "Sky", atau "Smoke".

[Home](#)[Articles](#) ▾

Tubes Stima

Element Recipe Tree Generation

DFS ▾

Smoke

Generate Tree

☐ Show live update

[Start generating trees...](#)

Gambar 3. Input teks

- Algorithm: Dropdown untuk memilih algoritma pencarian, yaitu BFS atau DFS.

[Home](#)[Articles](#) ▾

Tubes Stima

Element Recipe Tree Generation

DFS ▾

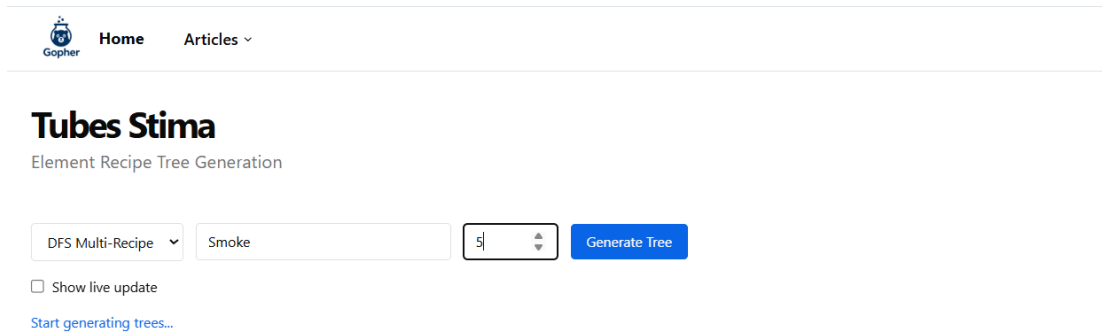
DFS
BFS
DFS Multi-Recipe
BFS Multi-Recipe

Smoke

Generate Tree

Gambar 4. Dropdown Algorithm choice

- Mode Pencarian: Opsi untuk memilih antara:
 - Multi Recipe → mencari beberapa solusi (jumlahnya dapat ditentukan),
 - Shortest Path → hanya mencari satu jalur terpendek.
- Max Paths: Input angka (aktif jika mode Multi Recipe).



Gopher Home Articles ▾

Tubes Stima

Element Recipe Tree Generation

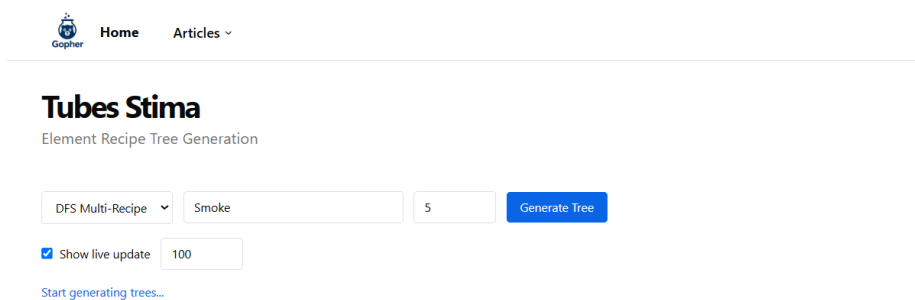
DFS Multi-Recipe ▾ Smoke 5 Generate Tree

☐ Show live update

[Start generating trees...](#)

Gambar 5. Input angka

- Live Update: Checkbox untuk mengaktifkan proses visualisasi secara real-time.
- Delay (ms): Input angka untuk mengatur kecepatan update per langkah (dalam milidetik).



Gopher Home Articles ▾

Tubes Stima

Element Recipe Tree Generation

DFS Multi-Recipe ▾ Smoke 5 Generate Tree

☒ Show live update 100

[Start generating trees...](#)

Gambar 6. Input angka live update

- Tombol Generate Tree: Menjalankan proses pencarian dengan parameter yang dimasukkan.
- Visualisasi Pohon Solusi:
 - Hasil pencarian ditampilkan dalam bentuk struktur pohon resep.
 - Visualisasi ini dihasilkan dari data tree yang diberikan backend.

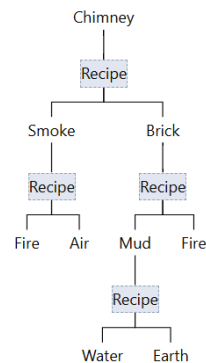
Tubes Stima

Element Recipe Tree Generation

BFS ▾ chimney Generate Tree

☐ Show live update

Nodes Explored: 6
Complete Paths: 1
Execution Time: 0 ms



Gambar 7. contoh hasil visualisasi

4.4.2. Fitur-Fitur yang Disediakan Program

Fitur	Deskripsi
Pencarian berbasis Graph	Menggunakan struktur graf dan pohon untuk merepresentasikan hubungan antar elemen.
Pemilihan Algoritma	Mendukung dua jenis algoritma pencarian: Breadth First Search (BFS) dan Depth First Search (DFS).
Dua Mode Pencarian	Tersedia mode Multi Recipe (banyak jalur) dan Shortest Path (jalur tercepat).
Live Visualization	Menampilkan progres pencarian secara bertahap saat live update diaktifkan.
Scraping Otomatis	Data elemen dan resep diambil langsung dari website Little Alchemy 2.
Penyesuaian Delay	Pengguna dapat mengatur kecepatan visualisasi (delay per langkah) dalam milidetik.

Routing API Backend	Backend Go menyediakan endpoint REST dan WebSocket untuk pemrosesan pencarian.
---------------------	--

4.4.3. Alur Penggunaan Program

1. Akses Aplikasi Melalui Browser
 - Pengguna membuka URL aplikasi melalui *browser*
<https://p01--app-frontend--7jx42hjyr69l.code.run/>
 - Halaman utama akan menampilkan form input pencarian elemen dan opsi parameter algoritma.
2. Mengisi Elemen Target dan Parameter
 - Pengguna memasukkan nama elemen tujuan di kolom Target Element. Contoh: "Brick", "Sky", atau "Smoke".
 - Pengguna juga dapat menentukan:
 - Jumlah maksimal recipe (khusus mode Multi Recipe).
 - Delay dalam milidetik jika ingin animasi Live Update berjalan lambat/cepat.
3. Memilih Algoritma dan Mode Pencarian
 - Pengguna memilih satu dari dua algoritma:
 - Breadth First Search (BFS): pencarian menyebar per level.
 - Depth First Search (DFS): pencarian mendalam ke satu cabang terlebih dahulu.
 - Kemudian pengguna memilih salah satu mode:
 - Multi Recipe: mencari beberapa jalur kombinasi hingga jumlah maksimum tercapai.
 - Shortest Path: mencari hanya satu jalur tercepat dari elemen dasar ke elemen target.
4. Menekan Tombol "Generate Tree"
 - Setelah semua parameter terisi, pengguna mengklik tombol generate tree.
 - Permintaan dikirim ke server backend Go melalui endpoint API (/api/bfs, /api/dfs, dst.).
5. Proses Pencarian dan Visualisasi
 - Backend memproses pencarian dan mengembalikan hasil dalam bentuk pohon solusi.
 - Terdapat dua skenario tampilan hasil:
 - Jika Live Update aktif:
 - Setiap langkah pencarian akan ditampilkan secara bertahap menggunakan WebSocket.
 - Pohon solusi akan tumbuh seiring pencarian berlangsung.
 - Jika Live Update nonaktif:
 - Pengguna menunggu proses selesai, lalu seluruh pohon hasil ditampilkan sekaligus.

6. Eksplorasi Lebih Lanjut

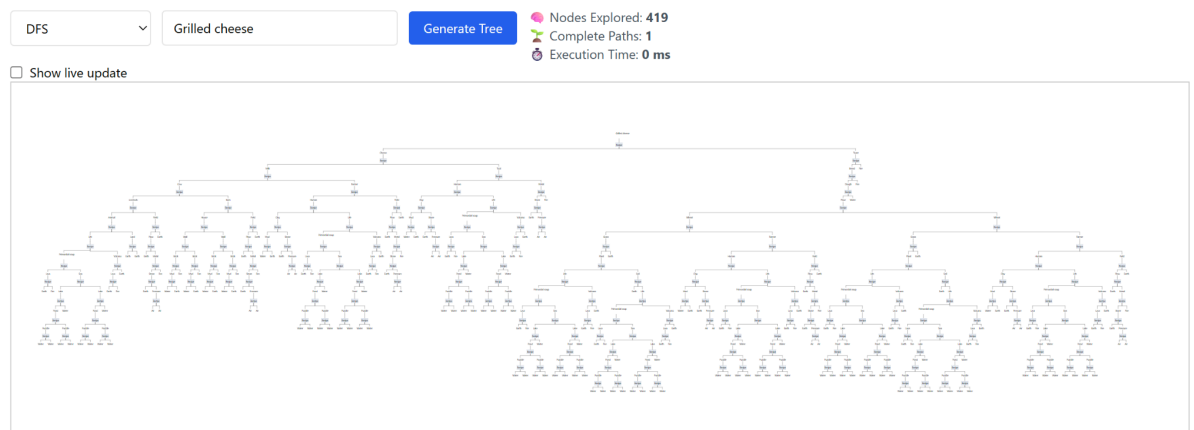
- Pengguna dapat mengganti elemen target, mengubah algoritma atau mode, lalu mengklik Start lagi untuk mencoba hasil yang berbeda.
- Tidak perlu reload halaman karena seluruh antarmuka mendukung interaksi dinamis.

4.5 Hasil pengujian

4.5.1. DFS One Recipe (Element: Grilled Cheese)

Tubes Stima

Element Recipe Tree Generation



4.5.2. DFS One Recipe (Element: Sky)

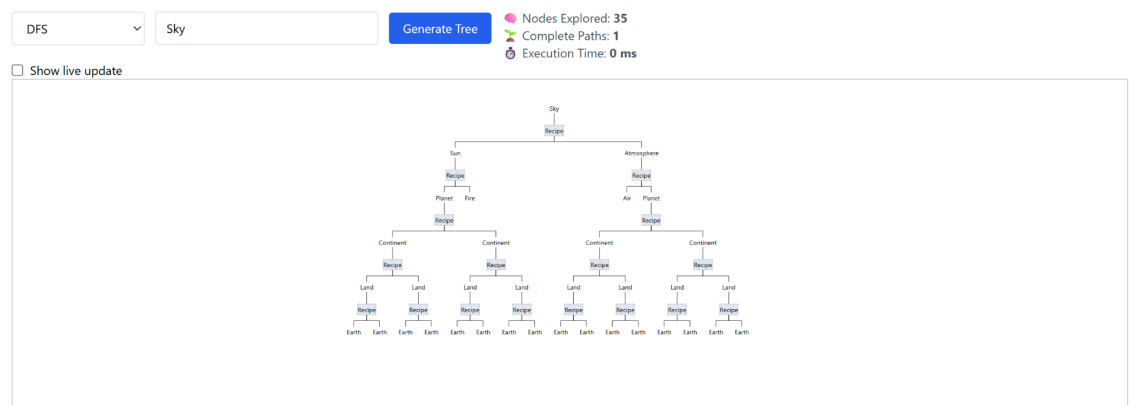


Home

Articles

Tubes Stima

Element Recipe Tree Generation



4.5.3. BFS One Recipe (Element: Sky)

[Home](#) [Articles](#) ▼

Tubes Stima

Element Recipe Tree Generation

BFS ▼

Sky

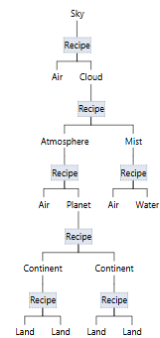
Generate Tree

Nodes Explored: 127

Complete Paths: 1

Execution Time: 0 ms

☐ Show live update



4.5.4. DFS MultiRecipe (Element: Sky) dengan Jumlah Resep Dua

[Home](#) [Articles](#) ▼

Tubes Stima

Element Recipe Tree Generation

DFS Multi-Recipe ▼

Sky

2

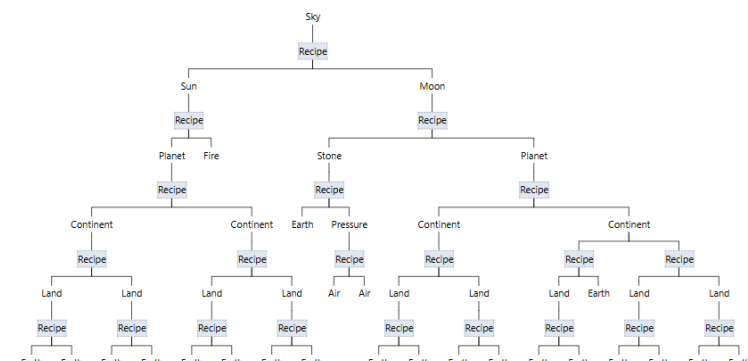
Generate Tree

Nodes Explored: 109

Complete Paths: 2

Execution Time: 0 ms

☐ Show live update



4.5.5. DFS MultiRecipe (Element: Sky) dengan Jumlah Resep Tiga

Tubes Stima

Element Recipe Tree Generation

[illegible]

4.5.6. BFS MultiRecipe (Element: WaterGun)

Tubes Stima

Element Recipe Tree Generation

BFS Multi-Recipe

Water gun

02

Generate Tree

Nodes Explored: 54
Complete Paths: 2
Execution Time: 0 ms

☐ Show live update

```

graph TD
    WaterGun[Water gun] --> Recipe1[Recipe]
    Recipe1 --> Gun[Gun]
    Recipe1 --> Water[Water]
    Gun --> Recipe2[Recipe]
    Recipe2 --> Bullet[Bullet]
    Recipe2 --> Metal1[Metal]
    Bullet --> Recipe3[Recipe]
    Recipe3 --> Gunpowder[Gunpowder]
    Recipe3 --> Metal2[Metal]
    Gunpowder --> Recipe4[Recipe]
    Recipe4 --> Energy[Energy]
    Recipe4 --> Dust[Dust]
    Energy --> Recipe5[Recipe]
    Recipe5 --> Fire1[Fire]
    Recipe5 --> Fire2[Fire]
    Dust --> Recipe6[Recipe]
    Recipe6 --> Land[Land]
    Recipe6 --> Air1[Air]
    Land --> Recipe7[Recipe]
    Recipe7 --> Earth1[Earth]
    Recipe7 --> Earth2[Earth]
    Metal2 --> Recipe8[Recipe]
    Recipe8 --> Stone[Stone]
    Recipe8 --> Heat[Heat]
    Stone --> Recipe9[Recipe]
    Recipe9 --> Air2[Air]
    Recipe9 --> Lava[Lava]
    Air2 --> Recipe10[Recipe]
    Recipe10 --> Earth3[Earth]
    Recipe10 --> Fire3[Fire]
    Lava --> Recipe11[Recipe]
    Recipe11 --> Earth4[Earth]
    Recipe11 --> Fire4[Fire]
    Heat --> Recipe12[Recipe]
    Recipe12 --> Air3[Air]
    Recipe12 --> Energy2[Energy]
    Air3 --> Recipe13[Recipe]
    Recipe13 --> Fire5[Fire]
    Recipe13 --> Fire6[Fire]
    Energy2 --> Recipe14[Recipe]
    Recipe14 --> Fire7[Fire]
    Recipe14 --> Fire8[Fire]

```

4.6 Analisis hasil pengujian

Bagian 4.5.1 menunjukkan bahwa algoritma DFS mampu menelusuri pohon resep hingga tier 15 dalam waktu yang sangat cepat, yakni hanya dalam hitungan milidetik, ketika digunakan untuk mencari satu resep Grilled Cheese. Hal ini membuktikan efisiensi DFS dalam eksplorasi mendalam meskipun struktur pohon cukup dalam. Sementara itu, hasil pada 4.5.2 dan 4.5.3 mengilustrasikan bahwa algoritma BFS dapat menemukan jalur pembuatan elemen Sky yang lebih pendek dibandingkan DFS, sesuai dengan sifat BFS

yang mengutamakan pencarian jalur optimal dalam hal jumlah langkah. Selanjutnya, 4.5.4 dan 4.5.5 memperlihatkan bahwa fitur *multi-recipe* pada DFS berfungsi dengan baik. Pada kasus dengan dua resep, sistem berhasil membangun dua jalur berbeda menuju elemen Sky. Namun, pada kasus tiga resep, hanya dua jalur yang ditemukan karena secara struktural, memang hanya terdapat dua kombinasi resep yang valid untuk menghasilkan elemen tersebut. Terakhir, hasil pada 4.5.6 menunjukkan bahwa pencarian *multi-recipe* menggunakan BFS terhadap elemen WaterGun berhasil dieksekusi dengan baik, membuktikan fleksibilitas dan ketahanan sistem dalam menangani elemen dengan banyak kemungkinan kombinasi.

BAB V

PENUTUP

5.1. Kesimpulan

Algoritma *Breadth First Search* (BFS) dan *Depth First Search* (DFS) yang diimplementasikan untuk pencarian recipe pada permainan Little Alchemy 2 menggunakan struktur data graf dan pohon secara efektif. Bfs menjamin solusi terpendek dengan kompleksitas waktu yang lebih tinggi, sementara DFS lebih hemat ruang namun tidak selalu optimal. Pendekatan ini relatif sederhana dan mudah diimplementasikan, dengan multithreading yang meningkatkan efisiensi pencarian multiple recipe. Aplikasi berhasil memenuhi spesifikasi, termasuk visualisasi pohon solusi, fitur Live Update, dan deployment menggunakan Docker.

5.2. Saran

Pengembangan lebih lanjut dapat mencakup optimasi algoritma, seperti penambahan fitur penyimpanan cache hasil pencarian juga dapat meningkatkan performa aplikasi.

5.3. Refleksi

Tugas Besar ini memberikan pemahaman mendalam tentang penerapan algoritma BFS dan DFS dalam menyelesaikan masalah nyata. Proses Pengembangan aplikasi web dengan Go untuk backend dan React/[Next.js](#) untuk frontend meningkatkan keterampilan teknis kami, terutama dalam hal concurrency dan arsitektur client-server. Tugas ini sangat membantu kami juga dalam pemahaman bagaimana suatu thread dapat digunakan untuk mengoptimalkan suatu algoritma. Meski demikian, terdapat juga tantangan-tantangan lain yang kami alami seperti berusaha untuk menyelesaikan tugas ini dengan beberapa asumsi, akibat kekurangmampuan kami dalam berusaha mengerti spesifikasi yang berubah setiap saat, sehingga kami harus mengadaptasikan code kami dengan perubahan tersebut dengan cepat.

LAMPIRAN

Github Repository : [Gopher Repozz](#)

Video Link : [Gopher Into You](#)

Deployment Link : [Gopher dot Com](#)

Tabel Poin

Poin	Ya	Tidak
1. Aplikasi dapat dijalankan.	✓	
2. Aplikasi dapat memperoleh data recipe melalui scraping.	✓	
3. Algoritma Depth First Search dan Breadth First Search dapat menemukan recipe elemen dengan benar.	✓	
4. Aplikasi dapat menampilkan visualisasi recipe elemen yang dicari sesuai dengan spesifikasi.	✓	
5. Aplikasi mengimplementasikan multithreading.	✓	
6. Membuat laporan sesuai dengan spesifikasi.	✓	
7. Membuat bonus video dan diunggah pada Youtube.	✓	
8. Membuat bonus algoritma pencarian Bidirectional.		✓
9. Membuat bonus Live Update.	✓	
10. Aplikasi di-containerize dengan Docker	✓	
11. Aplikasi di-deploy dan dapat diakses melalui internet.	✓	

DAFTAR PUSTAKA

- [1] R. Munir dan N. U. Maulidevi, *Breadth/Depth First Search (BFS/DFS)*, Bahan Kuliah IF2211 Strategi Algoritmik, Bagian 1. Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, 2025([https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/13-BFS-DFS-\(2025\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/13-BFS-DFS-(2025)-Bagian1.pdf))[Accessed: May. 12, 2025].
- [2] R. Munir dan N. U. Maulidevi, *Breadth/Depth First Search (BFS/DFS)*, Bahan Kuliah IF2211 Strategi Algoritmik, Bagian 2. Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, 2025([https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/14-BFS-DFS-\(2025\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/14-BFS-DFS-(2025)-Bagian2.pdf))[Accessed: May. 12, 2025].